UTM UNIVERSITI TEKNOLOGI MALAYSIA | Faculty of Computing

Semester 01 2025/2026

Subject : Database Programming (SECP3623)
Section : Section 2
Task : Project I
Due : 28th November 2025 (10%)

Name	Matric No.
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176

Instructions

Each group must submit **one (1)** project only.

Your submission must include:

1. **Project Report** in **PDF format** (named as LeaderName_GroupNum_ProjectI.pdf).
2. The **demo video URL** (YouTube or Google Drive link) **inside the report**.
    - Make sure the link is **working**, **publicly accessible**, and does **not require login**.
3. Include all SQL script files (.sql) by attaching them directly, compressing them into one zip folder, or providing a valid OneDrive sharing link inside the report.

TABLE OF CONTENTS

INTRODUCTION....3
A. DATA CREATION (DDL)....4
B. DATA MANIPULATION (DML)....7
C. DATA RETRIEVAL (DQL/SELECT)....11
D. Indexing and Optimization....23
TEAM WORK....25
CONCLUSION....27

INTRODUCTION

In many organisations and educational institutions, tracking attendance is an essential operational responsibility that ensures accountability, productivity and proper record management. Manual sign-in sheets and other traditional attendance methods frequently result in mistakes, redundancy and trouble accessing past data. In order to get over these limitations, automated attendance systems with dependable database backends are frequently used to increase data accuracy and quicken daily monitoring. An attendance system is a crucial part of efficient administrative operations since it usually maintains timetables, reporting tools, attendance records and student or staff data.

Our Attendance System's primary goal is to show how SQL can support all of the backend functionality needed to record attendance, update records, verify user-related data and retrieve data for reporting needs. By automating data handling, guaranteeing consistency across records and providing customizable queries like finding absentees , seeing attendance summaries and filtering records by date or category, the system wants to simplify the attendance process.

Furthermore, the database backend's goal is to create an organized MySQL database using SQL programming that effectively handles all attendance-related tasks. This requires creating a database with appropriate constraints, carrying out data transformation tasks, putting relevant queries with filtering and aggregating into practice and using indexing to boost efficiency. The Attendance System demonstrates how SQL may provide a solid basis for a thorough, safe and effective data management system with these backend solutions.

Table 1 shows the list of our team members and their assigned roles.

Table 1 Team members' name and roles

Name	Roles
Dayang Farah Farzana binti Abang Idham	- Lead on Filtering, Sorting and Aggregation - Complete Conclusion
Farra Nurzahin binti Zaharil Anuar	- Complete Introduction - Lead on Subqueries, Set Operations and Joins
Nurul Adriana binti Kamal Jefri	- Lead on Grouping and Filtering groups, Numeric and String Functions and Conditional Logic - Complete Indexing and Optimization
Safiya Nursyahadah binti Masnoor	- Lead on Database Creation (DDL) and Data Manipulation (DML)

A. DATA CREATION (DDL)

1. Create a database

- Creates a new database named attendance_system.
- USE selects this database so all following tables and queries are created/run inside it.

```
#A. DATABASE CREATION (DDL)
# 1. Create database
CREATE DATABASE attendance_system;
USE attendance_system;
```

2. Create Tables

2.1 students table

- Stores student information.
- student_id is the PRIMARY KEY (each student is unique).
- matric_no is NOT NULL and UNIQUE → no duplicate matric numbers.
- status has:
  - DEFAULT 'ACTIVE'
  - CHECK (status IN ('ACTIVE','INACTIVE')) → only valid statuses are allowed.

```
# 2. Create table
# 2.1 Table Students
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    matric_no VARCHAR(15) NOT NULL UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    programme VARCHAR(50) NOT NULL,
    status VARCHAR(10) NOT NULL DEFAULT 'ACTIVE'
        CHECK (status IN ('ACTIVE', 'INACTIVE'))
```

2.2 lecturers table

- Stores lecturer information.
- lecturer_id is PRIMARY KEY.
- staff_no is NOT NULL and UNIQUE.
- email also uses UNIQUE to avoid duplicates.

```
# 2.2 Table Lecturers
CREATE TABLE lecturers (
    lecturer_id INT PRIMARY KEY,
    staff_no VARCHAR(15) NOT NULL UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    office VARCHAR(50)
);
```

2.3 courses table

- Stores course information.
- course_id is PRIMARY KEY.
- course_code is UNIQUE so no duplicate course codes.
- credit_hour has a CHECK constraint to keep values between 1 and 6.
- lecturer_id is a FOREIGN KEY referring to lecturers(lecturer_id). This links each course to the lecturer who teaches it.

```
#2.3 Table Courses
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_code VARCHAR(10) NOT NULL UNIQUE,
    course_name VARCHAR(100) NOT NULL,
    credit_hour TINYINT NOT NULL CHECK (credit_hour BETWEEN 1 AND 6),
    lecturer_id INT NOT NULL,
    CONSTRAINT fk_course_lecturer FOREIGN KEY (lecturer_id) REFERENCES lecturers(lecturer_id)
);
```

2.4 enrollments table

- Represents which student takes which course.
- enrollment_id is PRIMARY KEY.
- student_id is a FOREIGN KEY → students(student_id).
- course_id is a FOREIGN KEY → courses(course_id).
- enrollment_date has DEFAULT (CURRENT_DATE).
- UNIQUE (student_id, course_id) to prevent the same student from enrolling in the same course twice.

```
#2.4 Table Enrollments
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enrollment_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    CONSTRAINT fk_enroll_student FOREIGN KEY (student_id) REFERENCES students(student_id),
    CONSTRAINT fk_enroll_course FOREIGN KEY (course_id) REFERENCES courses(course_id),
    CONSTRAINT uq_enroll_student_course UNIQUE (student_id, course_id)
);
```

2.5 class_sessions table

- Represents each class session (lesson) for a course.
- session_id is PRIMARY KEY.
- course_id is FOREIGN KEY → courses(course_id).
- CHECK (end_time > start_time) ensures the time range is valid.
- UNIQUE (course_id, session_date, start_time) to prevents duplicate sessions for the same course, date and time.

```sql
#2.5 Table Class Sessions
CREATE TABLE class_sessions (
    session_id INT PRIMARY KEY,
    course_id INT NOT NULL,
    session_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    CONSTRAINT fk_session_course FOREIGN KEY (course_id) REFERENCES courses(course_id),
    CONSTRAINT chk_time_order CHECK (end_time > start_time),
    CONSTRAINT uq_session_course_time UNIQUE (course_id, session_date, start_time)
```

2.6 attendance_records table

- Stores attendance status for each student in each session.
- attendance_id is PRIMARY KEY.
- enrollment_id is FOREIGN KEY → enrollments(enrollment_id).
- session_id is FOREIGN KEY → class_sessions(session_id).
- status is an ENUM with allowed values 'Present','Absent','Late','Excused' and default 'Absent'.
- recorded_at has DEFAULT CURRENT_TIMESTAMP.
- UNIQUE (enrollment_id, session_id) to ensure only one attendance record per student per session.

```sql
#2.6 Table Attendance Records
CREATE TABLE attendance_records (
    attendance_id INT PRIMARY KEY,
    enrollment_id INT NOT NULL,
    session_id INT NOT NULL,
    status ENUM('Present', 'Absent', 'Late', 'Excused') NOT NULL DEFAULT 'Absent',
    remarks VARCHAR(255),
    recorded_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_att_enrollment FOREIGN KEY (enrollment_id) REFERENCES enrollments(enrollment_id),
    CONSTRAINT fk_att_session FOREIGN KEY (session_id) REFERENCES class_sessions(session_id),
    CONSTRAINT uq_attendance_unique UNIQUE (enrollment_id, session_id)
```

3. ALTER table

- Modifies the existing students table to add a new column phone_no without recreating the table.

```sql
#3. ALTER TABLE students
ALTER TABLE students
ADD COLUMN phone_no VARCHAR (15) NULL AFTER email;
```

4. DROP table

- CREATE TABLE ... LIKE makes a backup table with the same structure as attendance_records.
- DROP TABLE IF EXISTS removes it if it exists.

```sql
#4. DROP TABLE
CREATE TABLE backup_attendance LIKE attendance_records;
DROP TABLE IF EXISTS backup_attendance;
```

B. DATA MANIPULATION (DML)

1. INSERT INTO

1.1 INSERT into students

- Inserts 11 student records which are matric number, full name, email, programme, status, and phone number.

```
INSERT INTO students (student_id, matric_no, full_name, email, programme, status, phone_no)
VALUES
(101, 'A23CS0001', 'Ali bin Abu', 'aliabu@graduate.utm.my', 'SECPH', 'ACTIVE', '011-12345678'),
(102, 'A23CS0002', 'Hafiz bin Masnoor', 'hafiz@graduate.utm.my', 'SECBH', 'ACTIVE', '012-1234588'),
(103, 'A23CS0003', 'Anwar bin Ibrahim', 'anwar@graduate.utm.my', 'SECVH', 'ACTIVE', '013-3334588'),
(104, 'A23CS0004', 'Siti binti Ali', 'siti@graduate.utm.my', 'SECRH', 'ACTIVE', '017-1237734'),
(105, 'A23CS0005', 'Aina binti Abdul', 'aina@graduate.utm.my', 'SECJH', 'ACTIVE', '012-2225555'),
(106, 'A23CS0006', 'Ismail bin Mail', 'mail@graduate.utm.my', 'SECPH', 'ACTIVE', '016-7783465'),
(107, 'A23CS0007', 'Shi Yu Qi', 'shiyuqi@graduate.utm.my', 'SECBH', 'ACTIVE', '014-9087669'),
(108, 'A23CS0008', 'Baek Ha Na', 'baekhana@graduate.utm.my', 'SECRH', 'ACTIVE', '011-90873352'),
(109, 'A23CS0009', 'Dhabitah binti Sabri', 'dhabitah@graduate.utm.my', 'SECVH', 'ACTIVE', '015-9078222'),
(110, 'A23CS0010', 'Aqmal bin Wahab', 'aqmal@graduate.utm.my', 'SECJH', 'ACTIVE', '010-2465555'),
(111, 'A23CS0011', 'Alia binti Kasim', 'alia@graduate.utm.my', 'SECPH', 'INACTIVE', '019-7684325');
```

student_id	matric_no	full_name	email	phone_no	programme	status
101	A23CS0001	Ali bin Abu	aliabu@graduate.utm.my	011-12345678	SECPH	ACTIVE
102	A23CS0002	Hafiz bin Masnoor	hafiz@graduate.utm.my	012-1234588	SECBH	ACTIVE
103	A23CS0003	Anwar bin Ibrahim	anwar@graduate.utm.my	013-3334588	SECVH	ACTIVE
104	A23CS0004	Siti binti Ali	siti@graduate.utm.my	017-1237734	SECRH	ACTIVE
105	A23CS0005	Aina binti Abdul	aina@graduate.utm.my	012-2225555	SECJH	ACTIVE
106	A23CS0006	Ismail bin Mail	mail@graduate.utm.my	016-7783465	SECPH	ACTIVE
107	A23CS0007	Shi Yu Qi	shiyuqi@graduate.utm.my	014-9087669	SECBH	ACTIVE
108	A23CS0008	Baek Ha Na	baekhana@graduate.utm.my	011-90873352	SECRH	ACTIVE
109	A23CS0009	Dhabitah binti Sabri	dhabitah@graduate.utm.my	015-9078222	SECVH	ACTIVE
110	A23CS0010	Aqmal bin Wahab	aqmal@graduate.utm.my	010-2465555	SECJH	ACTIVE
111	A23CS0011	Alia binti Kasim	alia@graduate.utm.my	019-7684325	SECPH	INACTIVE
NULL	NULL	NULL	NULL	NULL	NULL	NULL

1.2 INSERT into lecturers

- Inserts 11 lecturers with staff number, name, email, and office.

```
INSERT INTO lecturers (lecturer_id, staff_no, full_name, email, office)
VALUES
(201, 'L001', 'Dr. Rahim', 'rahim@utm.my', 'D-201'),
(202, 'L002', 'Dr. Nurul', 'nurul@utm.my', 'D-202'),
(203, 'L003', 'Prof. Kamal', 'kamal@utm.my', 'D-203'),
(204, 'L004', 'Dr. Faridah', 'faridah@utm.my', 'D-204'),
(205, 'L005', 'Dr. Haziq', 'haziq@utm.my', 'D-205'),
(206, 'L006', 'Dr. Ling Ling', 'lingling@utm.my', 'D-206'),
(207, 'L007', 'Dr. Aisha', 'aisha@utm.my', 'D-207'),
(208, 'L008', 'Dr. Mohan', 'mohan@utm.my', 'D-208'),
(209, 'L009', 'Dr. Sulaiman', 'sulaiman@utm.my', 'D-209'),
(210, 'L010', 'Dr. Farhan', 'farhan@utm.my', 'D-210'),
(211, 'L011', 'Dr. Faiq', 'faiq@utm.my', 'D-211');
```

lecturer_id	staff_no	full_name	email	office
201	L001	Dr. Rahim	rahim@utm.my	D-201
202	L002	Dr. Nurul	nurul@utm.my	D-202
203	L003	Prof. Kamal	kamal@utm.my	D-203
204	L004	Dr. Faridah	faridah@utm.my	D-204
205	L005	Dr. Haziq	haziq@utm.my	D-205
206	L006	Dr. Ling Ling	lingling@utm.my	D-206
207	L007	Dr. Aisha	aisha@utm.my	D-207
208	L008	Dr. Mohan	mohan@utm.my	D-208
209	L009	Dr. Sulaiman	sulaiman@utm.my	D-209
210	L010	Dr. Farhan	farhan@utm.my	D-210
211	L011	Dr. Faiq	faiq@utm.my	D-211
NULL	NULL	NULL	NULL	NULL

1.3 INSERT into courses

- Inserts 11 courses, each linked to a lecturer via lecturer_id.
- Uses different credit hours, which match the CHECK constraint.

```
INSERT INTO courses (course_id, course_code, course_name, credit_hour, lecturer_id)
VALUES
(301, 'SECJ101', 'Programming 1', 3, 201),
(302, 'SECI102', 'Programming 2', 3, 202),
(303, 'SECD103', 'Database', 4, 203),
(304, 'SECP104', 'Operating Systems', 3, 204),
(305, 'SECR105', 'Data Structures', 4, 205),
(306, 'SECD106', 'Computer Networks', 3, 206),
(307, 'SECV107', 'Web Development', 3, 207),
(308, 'SECI108', 'Software Engineering', 3, 208),
(309, 'SECB109', 'Cyber Security', 3, 209),
(310, 'SECV110', 'Machine Learning', 4, 210),
(311, 'SECR111', 'Artificial Intelligence', 4, 211);
```

course_id	course_code	course_name	credit_hour	lecturer_id
301	SECJ101	Programming 1	3	201
302	SECI102	Programming 2	3	202
303	SECD103	Database	4	203
304	SECP104	Operating Systems	3	204
305	SECR105	Data Structures	4	205
306	SECD106	Computer Networks	3	206
307	SECV107	Web Development	3	207
308	SECI108	Software Engineering	3	208
309	SECB109	Cyber Security	3	209
310	SECV110	Machine Learning	4	210
311	SECR111	Artificial Intelligence	4	211
NULL	NULL	NULL	NULL	NULL

1.4 INSERT into enrollments

- Inserts 11 enrollment records, mapping students to courses on specific dates.
- Populates the relationship between students and courses.

```
INSERT INTO enrollments (enrollment_id, student_id, course_id, enrollment_date)
VALUES
(401, 101, 301, '2025-01-10'),
(402, 102, 302, '2025-01-10'),
(403, 103, 303, '2025-01-10'),
(404, 104, 304, '2025-01-10'),
(405, 105, 305, '2025-01-11'),
(406, 106, 306, '2025-01-11'),
(407, 107, 307, '2025-01-11'),
(408, 108, 308, '2025-01-12'),
(409, 109, 309, '2025-01-12'),
(410, 110, 310, '2025-01-12'),
(411, 111, 311, '2025-01-12');
```

enrollment_id	student_id	course_id	enrollment_date
401	101	301	2025-01-10
402	102	302	2025-01-10
403	103	303	2025-01-10
404	104	304	2025-01-10
405	105	305	2025-01-11
406	106	306	2025-01-11
407	107	307	2025-01-11
408	108	308	2025-01-12
409	109	309	2025-01-12
410	110	310	2025-01-12
411	111	311	2025-01-12
NULL	NULL	NULL	NULL

1.5 INSERT into class_sessions

- Inserts 11 class sessions for different courses, dates, and times. These represent the actual sessions where attendance is taken.

```sql
INSERT INTO class_sessions (session_id, course_id, session_date, start_time, end_time)
VALUES
(501, 301, '2025-03-01', '09:00', '11:00'),
(502, 302, '2025-03-02', '10:00', '12:00'),
(503, 303, '2025-03-03', '11:00', '13:00'),
(504, 304, '2025-03-04', '09:00', '11:00'),
(505, 305, '2025-03-05', '08:00', '10:00'),
(506, 306, '2025-03-06', '13:00', '15:00'),
(507, 307, '2025-03-07', '14:00', '16:00'),
(508, 308, '2025-03-08', '09:00', '11:00'),
(509, 309, '2025-03-09', '11:00', '13:00'),
(510, 310, '2025-03-10', '15:00', '17:00'),
(511, 311, '2025-03-09', '11:00', '13:00');
```

session_id	course_id	session_date	start_time	end_time
501	301	2025-03-01	09:00:00	11:00:00
502	302	2025-03-02	10:00:00	12:00:00
503	303	2025-03-03	11:00:00	13:00:00
504	304	2025-03-04	09:00:00	11:00:00
505	305	2025-03-05	08:00:00	10:00:00
506	306	2025-03-06	13:00:00	15:00:00
507	307	2025-03-07	14:00:00	16:00:00
508	308	2025-03-08	09:00:00	11:00:00
509	309	2025-03-09	11:00:00	13:00:00
510	310	2025-03-10	15:00:00	17:00:00
511	311	2025-03-09	11:00:00	13:00:00
NULL	NULL	NULL	NULL	NULL

1.6 INSERT into attendance_records

- Inserts 11 attendance records, one per enrollment_id and session_id.
- Uses different status values (Present, Late, Absent, Excused) and remarks.

```sql
INSERT INTO attendance_records (attendance_id, enrollment_id, session_id, status, remarks)
VALUES
(601, 401, 501, 'Present', 'On time'),
(602, 402, 502, 'Late', '10 minutes late'),
(603, 403, 503, 'Absent', 'Overslept'),
(604, 404, 504, 'Excused', 'Medical leave'),
(605, 405, 505, 'Present', NULL),
(606, 406, 506, 'Present', NULL),
(607, 407, 507, 'Late', 'Traffic jam'),
(608, 408, 508, 'Present', NULL),
(609, 409, 509, 'Absent', 'Overslept'),
(610, 410, 510, 'Present', 'On time'),
(611, 411, 511, 'Present', NULL);
```

attendance_id	enrollment_id	session_id	status	remarks	recorded_at
601	401	501	Present	On time	2025-11-24 13:34:58
602	402	502	Late	10 minutes late	2025-11-24 13:34:58
603	403	503	Absent	Overslept	2025-11-24 13:34:58
604	404	504	Excused	Medical leave	2025-11-24 13:34:58
605	405	505	Present	NULL	2025-11-24 13:34:58
606	406	506	Present	NULL	2025-11-24 13:34:58
607	407	507	Late	Traffic jam	2025-11-24 13:34:58
608	408	508	Present	NULL	2025-11-24 13:34:58
609	409	509	Absent	Overslept	2025-11-24 13:34:58
610	410	510	Present	On time	2025-11-24 13:34:58
611	411	511	Present	NULL	2025-11-24 13:34:58
NULL	NULL	NULL	NULL	NULL	NULL

2. UPDATE

- Update the email of one specific student (student_id = 101).
- Demonstrates UPDATE with a proper WHERE condition.
- Changes an attendance status from 'Absent' to 'Excused'.
- Adds a remark to indicate that the student submitted an MC.

```
UPDATE students
SET email = 'ali_new@graduate.utm.my'
WHERE student_id = 101;

UPDATE attendance_records
SET status = 'Excused', remarks = 'MC submitted'
WHERE attendance_id = 609;
```

Before UPDATE :

student_id	full_name	email
101	Ali bin Abu	aliabu@graduate.utm.my
NULL	NULL	NULL

After UPDATE :

student_id	full_name	email
101	Ali bin Abu	ali_new@graduate.utm.my
NULL	NULL	NULL

Before UPDATE :

attendance_id	enrollment_id	session_id	status	remarks
609	409	509	Absent	Overslept
NULL	NULL	NULL	NULL	NULL

After UPDATE :

attendance_id	enrollment_id	session_id	status	remarks
609	409	509	Excused	MC submitted
NULL	NULL	NULL	NULL	NULL

3. DELETE

- First deletes one specific attendance record (child row).
- Then deletes the related enrollment row (parent row).
- This order respects the foreign key constraint between attendance_records and enrollments.
- Demonstrates DELETE with WHERE to remove only selected data.
- These SELECT queries verify that the rows have been successfully deleted.

```sql
#3. DELETE
DELETE FROM attendance_records
WHERE attendance_id = 610;

DELETE FROM enrollments
WHERE enrollment_id = 410;
```

```sql
SELECT *
FROM attendance_records
WHERE attendance_id = 610;

SELECT *
FROM enrollments
WHERE enrollment_id = 410;
```

Empty output for attendance_records (attendance_id = 610) :

	attendance_id	enrollment_id	session_id	status	remarks	recorded_at
*	NULL	NULL	NULL	NULL	NULL	NULL

Empty output for enrollments (enrollment_id = 410):

	enrollment_id	student_id	course_id	enrollment_date
*	NULL	NULL	NULL	NULL

C. DATA RETRIEVAL (DQL/SELECT)

1.0 Filtering (WHERE, AND, OR, NOT, BETWEEN, LIKE, NULL)

1.1 Filtering by Condition and Range (AND, BETWEEN)

- Select staff number, name and office from the lecturers table
- Filter lecturers with staff numbers between L003 and L008
- Further filters to only those in offices D-203, D-204 or D-205
- Displays the results sorted by staff number

```sql
SELECT
    staff_no
    full_name,
    office
FROM
    lecturers
WHERE
    staff_no BETWEEN 'L003' AND 'L008' AND office IN ('D-203', 'D-204', 'D-205')
ORDER BY
    staff_no;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	office
L003	D-203
L004	D-204
L005	D-205

1.2 Filtering by Pattern and Status (LIKE, OR, NOT)

- Select matric number, name and programme from the students table

- Filters students who are not in the SECPH programme or whose names contain “bin” or “binti”
- Shows the results sorted alphabetically by full name

```
228 •  SELECT
229        matric_no,
230        full_name,
231        programme
232    FROM
233        students
234    WHERE
235        NOT programme = 'SECPH'
236        OR full_name LIKE '%bin%'
237        OR full_name LIKE '%binti%'
238    ORDER BY
239        full_name;
```

Result Grid | Filter Rows: | Export:

matric_no	full_name	programme
A23CS0005	Aina binti Abdul	SECJH
A23CS0001	Ali bin Abu	SECPH
A23CS0011	Alia binti Kasim	SECPH
A23CS0003	Anwar bin Ibrahim	SECVH
A23CS0010	Aqmal bin Wahab	SECJH
A23CS0008	Baek Ha Na	SECRH
A23CS0009	Dhabitah binti Sabri	SECVH
A23CS0002	Hafiz bin Masnoor	SECBH
A23CS0006	Ismail bin Mail	SECPH
A23CS0007	Shi Yu Qi	SECBH
A23CS0004	Siti binti Ali	SECRH

1.3 Filtering for Missing Data (IS NULL)

- Retrieves attendance ID, student name, status and remarks from attendance records
- Joins attendance_records with enrollments and students to get the related student names
- Filters results to show only records where remarks are NULL (empty or not provided)

```
242 •  SELECT
243        A.attendance_id, S.full_name AS Student_name, A.status, A.remarks
244    FROM
245        attendance_records A
246    JOIN
247        enrollments E ON A.enrollment_id = E.enrollment_id
248    JOIN
249        students S ON E.student_id = S.student_id
250    WHERE
251        A.remarks IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

attendance_id	Student_name	status	remarks
605	Aina binti Abdul	Present	NULL
606	Ismail bin Mail	Present	NULL
608	Baek Ha Na	Present	NULL
611	Alia binti Kasim	Present	NULL

2.0 Sorting (ORDER BY, LIMIT)

- Selects course code, name and credit hours from the courses table
- Sorts results by credit hour (highest first) and the by course code (A-Z)
- Limits the output to the top 3 courses

```
254 •   SELECT
255         course_code,
256         course_name,
257         credit_hour
258     FROM
259         courses
260     ORDER BY
261         credit_hour DESC, course_code ASC
262     LIMIT 3;
263
```

Result Grid | Filter Rows: | Expo

course_code	course_name	credit_hour
SECD103	Database	4
SECR105	Data Structures	4
SECR111	Artificial Intelligence	4

3.0 Aggregation(COUNT, SUM, AVG, MAX, MIN)

3.1 Count and Average (COUNT, AVG)
- Counts the total number of class sessions using COUNT(session_id)
- Calculates the average credit hour of courses using AVG(credit_hour)
- Retrieves these values from the class_sessions and courses tables

```
266 •   SELECT
267         COUNT(session_id) AS Total_Sessions_held,
268         AVG(credit_hour) AS Average_Course_Credit_Hour
269     FROM
270         class_sessions, courses;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_Sessions_held	Average_Course_Credit_Hour
121	3.3636

3.2 Maximum and Minimum (MAX, MIN)
- Finds the earliest enrollment date using MIN(enrollment_date)
- Finds the latest enrollment date using MAX(enrollment_date)
- Retrieves these values from the enrollments table

```
273 •  SELECT
274        MIN(enrollment_date) AS Earliest_Enrollment_Date,
275        MAX(enrollment_date) AS Latest_Enrollment_Date
276    FROM
277        enrollments;
278
279
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Earliest_Enrollment_Date	Latest_Enrollment_Date
2025-01-10	2025-01-12

3.3 SUM

- Calculates the total credit hours of all courses using SUM(credit_hour)
- Retrieves this value from the courses table

```
280 •  SELECT
281        SUM(credit_hour) AS Total_Credit_Hours_Offered
282    FROM
283        courses;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_Credit_Hours_Offered
37

4. Grouping and filtering groups (GROUP BY, HAVING).

```
# 4. Grouping and filtering groups (GROUP BY, HAVING).
SELECT
    programme,
    COUNT(student_id) AS Total_Students
FROM
    students
GROUP BY
    programme
HAVING
    COUNT(student_id) > 1
ORDER BY
    Total_Students DESC;
```

programme	Total_Students
SECPH	3
SECBH	2
SECVH	2
SECRH	2
SECJH	2

5. Numeric and string functions (ROUND, TRUNCATE, UPPER, LENGTH, CONCAT, SUBSTR).

5.1 Query 1: String Functions (UPPER, LENGTH, CONCAT, SUBSTR)

```
# 5. Numeric and string functions (ROUND, TRUNCATE, UPPER, LENGTH, CONCAT, SUBSTR).
    # Query 1: String Functions (UPPER, LENGTH, CONCAT, SUBSTR)
    SELECT
        CONCAT(matric_no, ' - ', full_name) AS Student_Profile,
        UPPER(email) AS Uppercase_Email,
        LENGTH(full_name) AS Name_Length,
        SUBSTR(matric_no, 1, 3) AS Batch_Prefix
    FROM
        students;
```

Student_Profile	Uppercase_Email	Name_Length	Batch_Prefix
A23CS0001 - Ali bin Abu	ALI_NEW@GRADUATE.UTM.MY	11	A23
A23CS0002 - Hafiz bin Masnoor	HAFIZ@GRADUATE.UTM.MY	17	A23
A23CS0003 - Anwar bin Ibrahim	ANWAR@GRADUATE.UTM.MY	17	A23
A23CS0004 - Siti binti Ali	SITI@GRADUATE.UTM.MY	14	A23
A23CS0005 - Aina binti Abdul	AINA@GRADUATE.UTM.MY	16	A23
A23CS0006 - Ismail bin Mail	MAIL@GRADUATE.UTM.MY	15	A23
A23CS0007 - Shi Yu Qi	SHIYUQI@GRADUATE.UTM.MY	9	A23
A23CS0008 - Baek Ha Na	BAEKHANA@GRADUATE.UTM.MY	10	A23
A23CS0009 - Dhabitah binti Sabri	DHABITAH@GRADUATE.UTM.MY	20	A23
A23CS0010 - Aqmal bin Wahab	AQMAL@GRADUATE.UTM.MY	15	A23
A23CS0011 - Alia binti Kasim	ALIA@GRADUATE.UTM.MY	16	A23

5.2 Query 2: Numeric Functions (ROUND, TRUNCATE)

```
# Query 2: Numeric Functions (ROUND, TRUNCATE)
SELECT
course_name,
credit_hour,
ROUND(credit_hour / 3.0, 2) AS Calculated_Value_Rounded,
TRUNCATE(credit_hour / 3.0, 0) AS Calculated_Value_Truncated
FROM
    courses;
```

course_name	credit_hour	Calculated_Value_Rounded	Calculated_Value_Truncated
Programming 1	3	1.00	1
Programming 2	3	1.00	1
Database	4	1.33	1
Operating Systems	3	1.00	1
Data Structures	4	1.33	1
Computer Networks	3	1.00	1
Web Development	3	1.00	1
Software Engineering	3	1.00	1
Cyber Security	3	1.00	1
Machine Learning	4	1.33	1
Artificial Intelligence	4	1.33	1

6. Conditional logic (CASE WHEN).

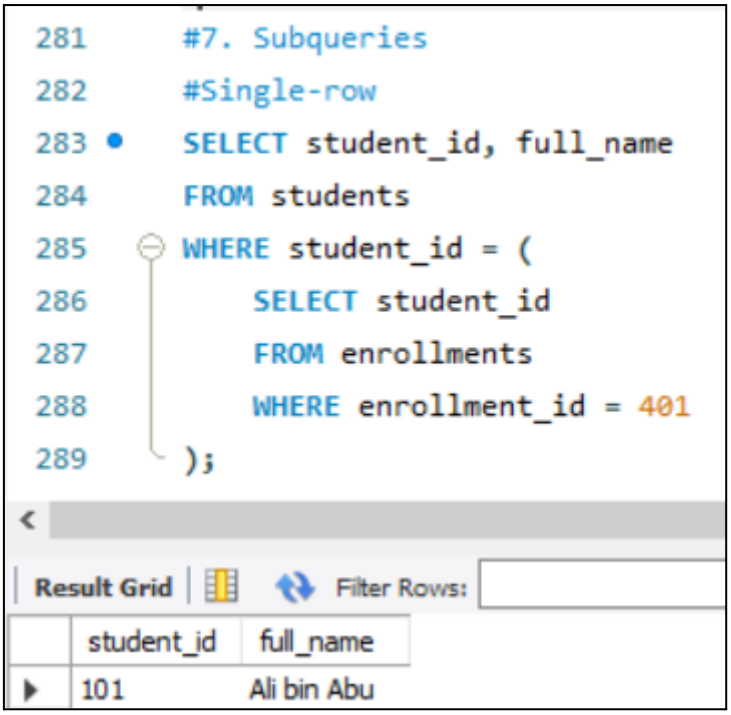
```sql
# 6. Conditional logic (CASE WHEN).
SELECT
    course_code,
    course_name,
    credit_hour,
    CASE
        WHEN credit_hour >= 4 THEN 'Intensive Course'
        WHEN credit_hour = 3 THEN 'Standard Course'
        ELSE 'Light Course'
    END AS Course_Intensity
FROM
    courses;
```

course_code	course_name	credit_hour	Course_Intensity
SECJ101	Programming 1	3	Standard Course
SECI102	Programming 2	3	Standard Course
SECD103	Database	4	Intensive Course
SECP104	Operating Systems	3	Standard Course
SECR105	Data Structures	4	Intensive Course
SECD106	Computer Networks	3	Standard Course
SECV107	Web Development	3	Standard Course
SECI108	Software Engineering	3	Standard Course
SECB109	Cyber Security	3	Standard Course
SECV110	Machine Learning	4	Intensive Course
SECR111	Artificial Intelligence	4	Intensive Course

7. Subqueries (single-row, multiple-row, correlated).

7.1 Single-row Subqueries

- Retrieve the specific student who is linked to enrollment ID 401.
- Uses a single-row subquery because enrollment_id is a PRIMARY KEY so it returns only one student_id.
- The outer query then displays the student's ID and full name.
- Relies on:
    - enrollment_id (PRIMARY KEY) to guarantee one result.
    - student_id (FOREIGN KEY) to ensure the student exists.



7.2 Multiple-row Subqueries

- Retrieve names of students enrolled in courses with 3 credit hours.
- Uses multiple-row subqueries, meaning the inner queries return lists of course_ids and student_ids.
- The IN operator is used because more than one matching value can exist.
- The inner query finds all courses with 3 credit hours then finds all students enrolled in those courses.
- Constraints involved:
  - course_id (PRIMARY KEY and FOREIGN KEY) to ensure valid matching
  - condition (WHERE) credit_hour to search for only students that enrolled in courses with a valid 3 credit hour value stored.
  - student_id (FOREIGN KEY) to ensure only real students appear.

```
291     # Multiple-row
292     SELECT full_name
293     FROM students
294     WHERE student_id IN (
295         SELECT student_id
296         FROM enrollments
297         WHERE course_id IN (
298             SELECT course_id
299             FROM courses
300             WHERE credit_hour = 3
301         )
302     );
```

Result Grid | Filter Rows:

full_name
Ali bin Abu
Hafiz bin Masnoor
Siti binti Ali
Ismail bin Mail
Shi Yu Qi
Baek Ha Na
Dhabitah binti Sabri

7.3 Correlated Subqueries

- Returns all students who have at least one ‘Absent’ attendance record.
- This is a correlated subquery, meaning it runs once per student and depends on the value of s.student_id.
- The subquery checks if there exists an attendance record marked ‘Absent’ linked to the same student.
- Constraints involved:
    - enrollment_id (PRIMARY KEY and FOREIGN KEY) to link attendance to enrollment.
    - condition (WHERE) status to valid attendance statuses equals to ‘Absent’ only.
    - student_id (FOREIGN KEY) in enrollments to ensure correct relational matching.

```sql
#Correlated
SELECT s.student_id, s.full_name
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM enrollments e
    JOIN attendance_records ar ON ar.enrollment_id = e.enrollment_id
    WHERE e.student_id = s.student_id
      AND ar.status = 'Absent'
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

student_id	full_name
103	Anwar bin Ibrahim

8. Set operations (UNION, NOT EXISTs).

8.1 UNION

- Combines student and lecturer names into one unified list.
- UNION automatically removes duplicate names.
- Works because both queries return the same number of columns with the same data type.
- Constraints involved:
    - full_name is NOT NULL to avoid invalid entries.
    - Matching data types between tables that are required for UNION.

```sql
#8. Set operations
#UNION
SELECT full_name
FROM students
UNION
SELECT full_name
FROM lecturers;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name
Ali bin Abu
Hafiz bin Masnoor
Anwar bin Ibrahim
Siti binti Ali
Aina binti Abdul
Ismail bin Mail
Shi Yu Qi
Baek Ha Na
Dhabitah binti Sabri
Aqmal bin Wahab
Alia binti Kasim
Dr. Rahim
Dr. Nurul
Prof. Kamal
Dr. Faridah
Dr. Haziq
Dr. Ling Ling
Dr. Aisha
Dr. Mohan
Dr. Sulaiman
Dr. Farhan
Dr. Faiq

8.2 NOT EXISTS

- Retrieves students who have no enrollment records.
- Uses NOT EXISTS to return students where no matching enrollment is found.
- Useful for identifying inactive students or new students not registered in any course
- Constraints involved:
    - student_id (PRIMARY KEY) in students for unique identification.
    - student_id (FOREIGN KEY) in enrollments to allow checking for missing records.

```
323     #NOT EXISTS
324 •   SELECT s.student_id, s.full_name
325     FROM students s
326     WHERE NOT EXISTS (
327         SELECT 1
328         FROM enrollments e
329         WHERE e.student_id = s.student_id
330     );
```

Result Grid | Filter Rows: | Edit:

student_id	full_name
110	Aqmal bin Wahab

9. Joins (NATURAL, INNER, LEFT OUTER, SELF).

9.1 Natural Join

- Automatically joins the two tables using column with the same name between the class_sessions table with the courses table.
- Both tables share the column course_id so the join is performed using that column.
- The result shows combined information for each class session and its related course details.
- NATURAL JOIN removes duplicate columns.
- Constraints involved:
    - course_id (PRIMARY KEY) in courses
    - course_id (FOREIGN KEY) in class_sessions to ensure correct linking.

```
#9. Joins
#NATURAL
SELECT *
FROM class_sessions
NATURAL JOIN courses;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

course_id	session_id	session_date	start_time	end_time	course_code	course_name	credit_hour	lecturer_id
301	501	2025-03-01	09:00:00	11:00:00	SECJ101	Programming 1	3	201
302	502	2025-03-02	10:00:00	12:00:00	SECI102	Programming 2	3	202
303	503	2025-03-03	11:00:00	13:00:00	SECD103	Database	4	203
304	504	2025-03-04	09:00:00	11:00:00	SECP104	Operating Systems	3	204
305	505	2025-03-05	08:00:00	10:00:00	SECR105	Data Structures	4	205
306	506	2025-03-06	13:00:00	15:00:00	SECD106	Computer Networks	3	206
307	507	2025-03-07	14:00:00	16:00:00	SECV107	Web Development	3	207
308	508	2025-03-08	09:00:00	11:00:00	SECI108	Software Engineering	3	208
309	509	2025-03-09	11:00:00	13:00:00	SECB109	Cyber Security	3	209
310	510	2025-03-10	15:00:00	17:00:00	SECV110	Machine Learning	4	210
311	511	2025-03-09	11:00:00	13:00:00	SECR111	Artificial Intelligence	4	211

9.2 INNER JOIN

- Shows students along with the courses they are enrolled in.
- INNER JOIN returns only rows where all tables have matching records.
- The first join links students with enrollments using student_id.
- The second join links enrollments with courses using course_id.
- If a student has no enrollments or courses, they will not appear.
- Constraints involved:
    - student_id (FOREIGN KEY) in enrollments.
    - course_id (FOREIGN KEY) in enrollments.

```
#INNER
SELECT s.full_name, c.course_name
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	course_name
Ali bin Abu	Programming 1
Hafiz bin Masnoor	Programming 2
Anwar bin Ibrahim	Database
Siti binti Ali	Operating Systems
Aina binti Abdul	Data Structures
Ismail bin Mail	Computer Networks
Shi Yu Qi	Web Development
Baek Ha Na	Software Engineering
Dhabitah binti Sabri	Cyber Security
Alia binti Kasim	Artificial Intelligence

9.3 LEFT OUTER

- Returns all students, even if they have no enrollment or no attendance records.
- A LEFT OUTER JOIN returns all rows from the table students even if there is no matching record in the right tables (enrollments or attendance_records).
- Students without attendance will show NULL in the status column.
- Constraints involved:
    - student_id (FOREIGN KEY) in enrollments.
    - enrollment_id (FOREIGN KEY) in attendance_records.

```
344     #LEFT OUTER
345 •   SELECT s.student_id, s.full_name, ar.status
346     FROM students s
347     LEFT JOIN enrollments e ON s.student_id = e.student_id
348     LEFT JOIN attendance_records ar ON ar.enrollment_id = e.enrollment_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

student_id	full_name	status
101	Ali bin Abu	Present
102	Hafiz bin Masnoor	Late
103	Anwar bin Ibrahim	Absent
104	Siti binti Ali	Excused
105	Aina binti Abdul	Present
106	Ismail bin Mail	Present
107	Shi Yu Qi	Late
108	Baek Ha Na	Present
109	Dhabitah binti Sabri	Excused
110	Aqmal bin Wahab	NULL
111	Alia binti Kasim	Present

9.4 Self Join

- Compares rows from the same table to find students in the same programme.
- A SELF JOIN joins a table with itself and treats one copy as s1 and another as s2.
- Returns unique pairs because of the condition student_id < s2.student_id.
- Constraints involved:
    - student_id (PRIMARY KEY) to ensure unique ordering.
    - programme (NOT NULL) is required for meaningful matching.

```
350     #SELF
351 •   SELECT s1.full_name AS student1,
352            s2.full_name AS student2,
353            s1.programme
354     FROM students s1
355     JOIN students s2
356     ON s1.programme = s2.programme
357     AND s1.student_id < s2.student_id;
```

Result Grid | Filter Rows:

student1	student2	programme
Ali bin Abu	Ismail bin Mail	SECPH
Ali bin Abu	Alia binti Kasim	SECPH
Hafiz bin Masnoor	Shi Yu Qi	SECBH
Anwar bin Ibrahim	Dhabitah binti Sabri	SECVH
Siti binti Ali	Baek Ha Na	SECRH
Aina binti Abdul	Aqmal bin Wahab	SECJH
Ismail bin Mail	Alia binti Kasim	SECPH

D. Indexing and Optimization

● Create two indexes:

o BTREE (for numeric/date columns).

```
# D. Indexing and Optimization
# Create two indexes:
    # BTREE (for numeric/date columns).
    EXPLAIN SELECT * FROM courses WHERE credit_hour = 4;
    CREATE INDEX idx_credit_hour ON courses(credit_hour) USING BTREE;
    EXPLAIN SELECT * FROM courses WHERE credit_hour = 4;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	courses	NULL	ALL	NULL	NULL	NULL	NULL	11	10.00	Using where

Before

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	class_sessions	NULL	ref	idx_session_date	idx_session_date	3	const	1	100.00	NULL

After

o TEXT (for text search).

```
# TEXT (for text search).
EXPLAIN SELECT * FROM attendance_records WHERE remarks LIKE '%Medical%';
CREATE FULLTEXT INDEX idx_remarks ON attendance_records(remarks);
EXPLAIN SELECT * FROM attendance_records WHERE MATCH(remarks) AGAINST('Medical');
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	attendance_records	NULL	ALL	NULL	NULL	NULL	NULL	10	11.11	Using where

Before

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	attendance_records	NULL	fulltext	idx_remarks	idx_remarks	0	const	1	100.00	Using where; Ft_hints: sorted

After

- Analyse query plans before and after index creation.
- Write a short interpretation of what changed (rows examined, possible keys).

1. BTREE Index Analysis

- Before Indexing: The query performed a Full Table Scan (type: ALL). The database had to check every single row in the `class_sessions` table to find the specific date, which is inefficient as data grows.
- After Indexing: The query used the `idx_session_date` key. The search type changed to `ref` (or `range`), meaning the database went directly to the specific location in the B-Tree structure. The number of rows examined decreased, significantly optimizing retrieval time.

2. TEXT Index Analysis

- Before Indexing: Searching for text using `LIKE '%word%'` forced the database to read the entire `remarks` string for every student to see if it contained the pattern.
- After Indexing: By creating a FULLTEXT index and switching to `MATCH...AGAINST`, the database used an inverted index structure. This allows it to instantly locate records containing specific keywords (like "Medical") without scanning unrelated text, making text searches much faster on large datasets.

TEAM WORK

1. **Name :** Dayang Farah Farzana binti Abang Idham

   Describe Role : My role was focusing on the core DQL requirements of Filtering, Sorting and Aggregation. Specifically, I utilized the Filtering clauses (WHERE, AND, OR, NOT, BETWEEN, LIKE, NULL), Sorting clauses (ORDER BY, LIMIT) and Aggregation functions (COUNT, SUM, AVG, MAX, MIN).

   What I Learned : Through this process, I significantly deepened my understanding of how to use various filtering operators effectively to isolate specific data sets, such as using LIKE for pattern matching and BETWEEN for range selection. I also learned the practical application of aggregate functions such as calculating the average price (AVG) or finding the highest record (MAX) and how the LIMIT clause is essential for controlling the result size in sorted queries.

2. **Name :** Farra Nurzahin binti Zaharil Anuar

   Describe Role : I was responsible for writing SQL queries to retrieve and manipulate data from the attendance system database. My tasks included creating subqueries, set operations and joins to analyze relationships between students, courses and attendance records.

   What I Learned : I learned how to work with databases and SQL queries to retrieve and analyze information effectively. Specifically, I practiced using subqueries (single-row, multiple-row and correlated) to extract precise data, set operations like UNION and NOT EXISTS to combine filter results and different types of join (Natural, Inner, Left Outer and Self) to relate multiple tables. I also understood the role of constraints such as primary keys and foreign keys in ensuring accurate and consistent query results.

3. **Name :** Nurul Adriana Binti Kamal Jefri

   Describe Role : I was responsible for implementing Grouping and Filtering via GROUP BY and HAVING clauses to generate summary data. I also applied Numeric and String Functions (such as CONCAT, UPPER, and ROUND) to format query outputs and utilized Conditional Logic (CASE WHEN) to categorize data dynamically. Finally, I handled the Database Indexing and Optimization, where I implemented BTREE and FULLTEXT indexes to improve performance.

   What I Learned : I learned how to effectively aggregate data to uncover trends, specifically understanding the critical difference between filtering rows with WHERE versus filtering groups with HAVING. I gained practical skills in data formatting to ensure raw database values are presented in a user-friendly manner using string and numeric functions. Most importantly, I developed a deeper understanding of database performance; by using the EXPLAIN statement, I observed how indexing transforms a costly full-table scan into an efficient lookup, significantly reducing the "rows examined" during queries.

4. **Name :** Safiya Nursyahadah binti Masnoor

   Describe Role : I was responsible for the DDL and DML components of the project. My work involved creating all six tables required for the attendance system which were students, lecturers, courses, enrollments, class_sessions, and attendance_records. I applied a range of SQL constraints that included PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT. I also inserted more than ten records into each table and executed UPDATE and DELETE statements to modify and remove specific entries. In addition, I used ALTER TABLE and DROP TABLE to demonstrate how the database structure can be revised when necessary.

   What I Learned : Through this project I developed a clear understanding of how to design and construct a complete database structure using DDL commands, I gained practical experience in creating tables with appropriate constraints to maintain data accuracy and enforce relationships between tables. I also learned how to insert, update and delete records using DML statements with the correct condition. This project strengthened my understanding of database integrity and illustrated how foreign keys support relational consistency. It also shows how DDL and DML operate together to build a functional backend for an application.

CONCLUSION

In conclusion, the development of our Attendance System allowed us to apply a wide range of SQL concepts to design a functional, reliable and well-structured database for managing attendance records. Through creating tables, inserting and manipulating data, performing complex queries and analyzing indexing performance, we gained a deeper understanding of how SQL supports real-world data management. However, we faced several challenges such as ensuring correct relationships between tables, troubleshooting constraint errors and working with advanced SQL features like correlated subqueries and full-text indexing. These difficulties required careful planning and repeated testing to ensure accuracy and consistency across the system. To improve the system in the future, we suggest adding automated features such as stored procedure, triggers and role-based access control. Overall, this project strengthened our SQL proficiency and demonstrated the importance of a well-designed backend in supporting an effective attendance management system.

Presentation Link -
https://drive.google.com/file/d/1ZxQXE-LnqM2zygCzTehLCCId0qeS5vMl/view?usp=sharing